

Lab_05_Thomas.qmd

Thomas Toledo Travassos

###LAB_ESPECIAL

1. Carregando os packages que iremos utilizar temos;

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(readr)
library(leaflet)
library(lubridate)
```

2. Criando a Função analisa_aeronave temos;

```
analisa_aeronave <- function(tail_number, arquivo) {

  # Leitura do arquivo por partes
  voos <- list()

  callback <- DataFrameCallback$new(function(dados, pos) {

    dados_aeronave <- dados %>%
```

```

    filter(TAIL_NUMBER == tail_number)

    if (nrow(dados_aeronave) > 0) {
      voos[[length(voos) + 1]] <- dados_aeronave
    }
  })

read_csv_chunked(
  arquivo,
  callback = callback,
  chunk_size = 100000,
  progress = FALSE
)

# Junta os resultados
if (length(voos) == 0) {
  stop("Aeronave não encontrada.")
}

tabela_voos <- bind_rows(voos)

# Remove voos cancelados e registros sem aeroportos
tabela_voos <- tabela_voos %>%
  filter(
    CANCELLED == 0,
    !is.na(ORIGIN_AIRPORT),
    !is.na(DESTINATION_AIRPORT)
  )

# Converte horário de partida para número e cria data/hora
tabela_voos <- tabela_voos %>%
  mutate(
    DEPARTURE_TIME = as.numeric(DEPARTURE_TIME),
    hora = floor(DEPARTURE_TIME / 100),
    minuto = DEPARTURE_TIME %% 100,
    data_hora = make_datetime(
      YEAR,
      MONTH,
      DAY,

```

```

        hora,
        minuto
    )
) %>%
arrange(data_hora)

# Importa coordenadas dos aeroportos
arquivo_aeroportos <- file.path(
    dirname(arquivo),
    "airports.csv"
)

aeroportos <- read_csv(
    arquivo_aeroportos,
    show_col_types = FALSE
) %>%
select(
    IATA_CODE,
    LATITUDE,
    LONGITUDE
)

# Coordenadas dos aeroportos de origem
tabela_voos <- tabela_voos %>%
left_join(
    aeroportos %>%
        rename(
            ORIGIN_LATITUDE = LATITUDE,
            ORIGIN_LONGITUDE = LONGITUDE
        ),
    by = c("ORIGIN_AIRPORT" = "IATA_CODE")
)

# Coordenadas dos aeroportos de destino
tabela_voos <- tabela_voos %>%
left_join(
    aeroportos %>%
        rename(
            DESTINATION_LATITUDE = LATITUDE,

```

```

        DESTINATION_LONGITUDE = LONGITUDE
      ),
      by = c("DESTINATION_AIRPORT" = "IATA_CODE")
    )

# Calcula velocidade média
tabela_voos <- tabela_voos %>%
  mutate(
    AIR_TIME = as.numeric(AIR_TIME),
    DISTANCE = as.numeric(DISTANCE),
    VELOCIDADE_MEDIA = ifelse(
      !is.na(AIR_TIME) & AIR_TIME > 0,
      DISTANCE / (AIR_TIME / 60),
      NA
    )
  )

# Cria o mapa
mapa <- leaflet() %>%
  addTiles()

# Identifica os aeroportos utilizados
aeroportos_mapa <- bind_rows(
  tabela_voos %>%
    transmute(
      IATA_CODE = ORIGIN_AIRPORT,
      LATITUDE = ORIGIN_LATITUDE,
      LONGITUDE = ORIGIN_LONGITUDE
    ),

  tabela_voos %>%
    transmute(
      IATA_CODE = DESTINATION_AIRPORT,
      LATITUDE = DESTINATION_LATITUDE,
      LONGITUDE = DESTINATION_LONGITUDE
    )
) %>%
distinct(IATA_CODE, .keep_all = TRUE) %>%
filter(

```

```

    !is.na(LATITUDE),
    !is.na(LONGITUDE)
  )

# Adiciona aeroportos ao mapa
mapa <- mapa %>%
  addCircleMarkers(
    data = aeroportos_mapa,
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    radius = 5,
    popup = ~IATA_CODE
  )

# Adiciona os voos em ordem cronológica
for (i in seq_len(nrow(tabela_voos))) {

  voo <- tabela_voos[i, ]

  if (
    !is.na(voo$ORIGIN_LATITUDE) &&
    !is.na(voo$ORIGIN_LONGITUDE) &&
    !is.na(voo$DESTINATION_LATITUDE) &&
    !is.na(voo$DESTINATION_LONGITUDE)
  ) {

    mapa <- mapa %>%
      addPolylines(
        lng = c(
          voo$ORIGIN_LONGITUDE,
          voo$DESTINATION_LONGITUDE
        ),
        lat = c(
          voo$ORIGIN_LATITUDE,
          voo$DESTINATION_LATITUDE
        ),
        weight = ifelse(
          is.na(voo$VELOCIDADE_MEDIA),
          2,
          max(2, min(10, voo$VELOCIDADE_MEDIA / 100))
        )
      )
  }
}

```

```

    ),
    popup = paste0(
      voo$ORIGIN_AIRPORT,
      " → ",
      voo$DESTINATION_AIRPORT,
      "<br>Velocidade média: ",
      round(voo$VELOCIDADE_MEDIA, 2),
      " milhas/h"
    )
  )
}
}

# Retorna tabela e mapa
list(
  tabela = tabela_voos,
  mapa = mapa
)
}

```

3. Executando a Função temos;

```

resultado <- analisa_aeronave(
  tail_number = "N431WN",
  arquivo = "datasets/flights.csv"
)

```

```

-- Column specification -----
cols(
  .default = col_double(),
  AIRLINE = col_character(),
  TAIL_NUMBER = col_character(),
  ORIGIN_AIRPORT = col_character(),
  DESTINATION_AIRPORT = col_character(),
  SCHEDULED_DEPARTURE = col_character(),
  DEPARTURE_TIME = col_character(),
  WHEELS_OFF = col_character(),
  WHEELS_ON = col_character(),
  SCHEDULED_ARRIVAL = col_character(),
  ARRIVAL_TIME = col_character(),

```

```

    CANCELLATION_REASON = col_character()
  )
  i Use `spec()` for the full column specifications.

```

4. Tabela;

```
resultado$tabela
```

```

# A tibble: 1,926 x 39
  YEAR MONTH DAY DAY_OF_WEEK AIRLINE FLIGHT_NUMBER TAIL_NUMBER
  <dbl> <dbl> <dbl>      <dbl> <chr>          <dbl> <chr>
1  2015     1     1          4 WN             1762 N431WN
2  2015     1     1          4 WN             3564 N431WN
3  2015     1     1          4 WN             3564 N431WN
4  2015     1     1          4 WN             3564 N431WN
5  2015     1     2          5 WN             1793 N431WN
6  2015     1     2          5 WN             1793 N431WN
7  2015     1     2          5 WN             1793 N431WN
8  2015     1     2          5 WN             1793 N431WN
9  2015     1     2          5 WN             1793 N431WN
10 2015     1     2          5 WN             3144 N431WN
# i 1,916 more rows
# i 32 more variables: ORIGIN_AIRPORT <chr>, DESTINATION_AIRPORT <chr>,
#   SCHEDULED_DEPARTURE <chr>, DEPARTURE_TIME <dbl>, DEPARTURE_DELAY <dbl>,
#   TAXI_OUT <dbl>, WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>,
#   AIR_TIME <dbl>, DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>,
#   SCHEDULED_ARRIVAL <chr>, ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>,
#   DIVERTED <dbl>, CANCELLED <dbl>, CANCELLATION_REASON <chr>, ...

```

```

Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()

```

```
[1] "2026-09-16 19:34:47 -03"
```